PROGRAM: CYBERSECURITY ETHICAL HACKER

MENTOR UTAMA: Bagus Sudung Prakoso

PENYELENGGARA: PT VINIX SEVEN AURUM

Topik: Modul 9 - Cryptography, Reverse Engineering & Penetration Testing

Sandi Operasi: Operation Deadlock (Advanced CrackMe)

Kelompok 8 :

- Afnan Yazid Pradana
- Kelvin Febrianto
- Kurniawan Saputra
- Rishfan Mailizar
- Robin Felix Hama

Cybersecurity Ethical Hacker

Target Artefak: deadlock_crackme.elf (Linux Binary 64-bit)

(Catatan: Tersedia juga deadlock_crackme.exe bagi yang menganalisis di Windows)

A. DESKRIPSI TUGAS

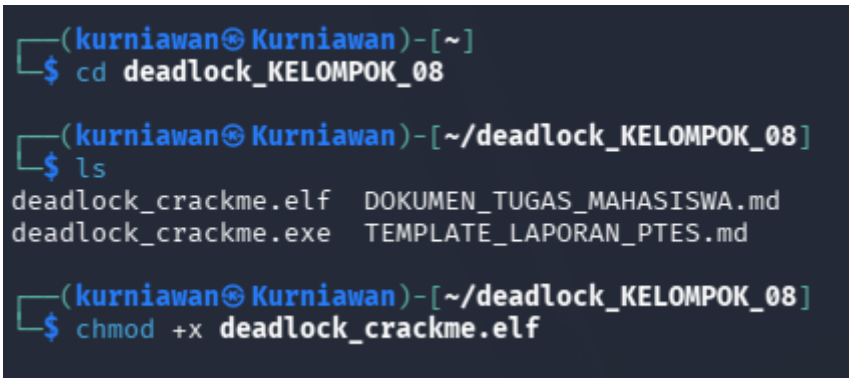
Sebagai Lead Penetration Tester, Anda ditugaskan untuk melakukan Reverse Engineering dan Cryptanalysis terhadap sebuah artefak binary mencurigakan berstatus "Zero-Day". Tidak ada source code yang diberikan. Seluruh simbol fungsi pada binary telah dihilangkan (stripped). Anda hanya dibekali satu paket executable terisolasi sesuai alokasi ID Anda (Kelompok atau Individu).

Tugas Anda adalah membedah artefak tersebut hingga ke tingkat Assembly (x86_64), melewati mekanisme proteksi anti-debugging, mengekstrak kunci dan ciphertext langsung dari RAM saat runtime, membalikkan logika kriptografi berantai (chained cipher), menyusun skrip otomasi dekripsi, serta menyusun laporan berstandar PTES (Penetration Testing Execution Standard).

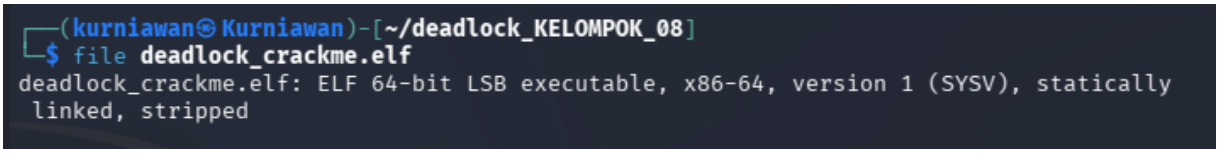
B. RINCIAN TUGAS (5 FASE EKSTREM)

BAGIAN 1: ADVANCED STATIC ANALYSIS & ANTI-DEBUGGING BYPASS (Bobot: 20%) Artefak ini dirancang dengan mekanisme pelindungan diri (anti-debugging) yang akan langsung menghentikan program jika mendeteksi adanya debugger (seperti GDB).

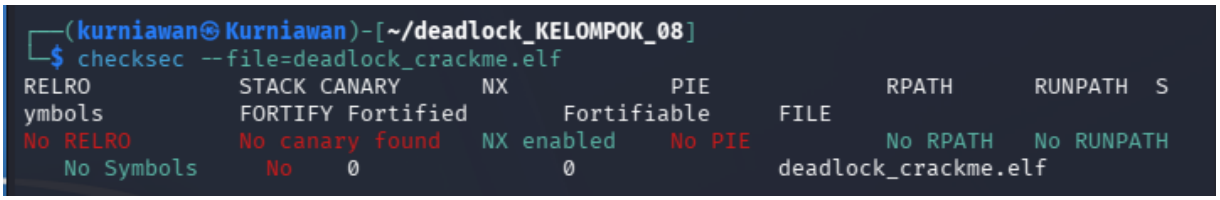
1. Disassembly & Function Tracing: Gunakan perangkat disassembler seperti Ghidra, IDA Pro, atau Radare2 untuk membedah instruksi Assembly (x86_64) dari binary tersebut. Karena binary berstatus stripped, lacak alur eksekusi dari entry point (_start) menuju fungsi utama main().



Hasil Gambar ini menunjukkan proses persiapan analisis file binary melalui terminal Linux. Perintah cd digunakan untuk masuk ke direktori kerja kelompok, kemudian perintah ls digunakan untuk melihat file yang tersedia, termasuk deadlock_crackme.elf dan deadlock_crackme.exe. Selanjutnya, perintah chmod +x deadlock_crackme.elf digunakan untuk memberikan izin eksekusi pada file binary Linux agar dapat dijalankan dalam tahap analisis berikutnya.



Hasil pemeriksaan file deadlock_crackme.elf menggunakan perintah file. Hasilnya mengidentifikasi file tersebut sebagai executable ELF 64-bit untuk arsitektur x86-64 yang dikompilasi secara statis dan berstatus stripped. Informasi ini penting untuk menentukan lingkungan analisis yang sesuai serta menunjukkan bahwa simbol fungsi telah dihilangkan, sehingga penelusuran instruksi program membutuhkan bantuan perangkat analisis binary.



Maksud Perintah: Perintah checksec adalah utilitas keamanan untuk memeriksa fitur

perlindungan biner (binary protections) yang aktif atau tidak aktif pada suatu file program. Hal ini penting bagi pentester atau reverser untuk mengetahui tingkat kesulitan eksploitasi atau proteksi memori apa saja yang terpasang.

Hasil Analisis Output:

- No RELRO (Relocation Read-Only): Tabel memori program tidak dilindungi dari modifikasi penulisan (mudah untuk melakukan teknik *GOT overriding* jika diperlukan).
- No canary found (Stack Canary): Tidak ada pengaman *stack canary* yang mendeteksi kerusakan pada tumpukan memori (*stack overflow*).
- NX enabled (No-Execute): Memori bagian *stack* ditandai tidak bisa dieksekusi (mencegah injeksi *shellcode* langsung di stack).
- No PIE (Position Independent Executable): Alamat memori dasar program bersifat statis (tidak acak), sehingga alamat fungsi atau variabel di dalam memori akan selalu tetap setiap kali program dijalankan.

2. Anti-Analysis Defeat: Identifikasi instruksi pemanggilan proteksi anti-debugging (ptrace pada Linux atau IsDebuggerPresent pada Windows). Lakukan patching instruksi (memodifikasi nilai HEX dari instruksi Jump, misalnya mengubah JNE menjadi JMP atau NOP) agar binary tersebut bisa dijalankan di dalam debugger tanpa mengalami exit otomatis.

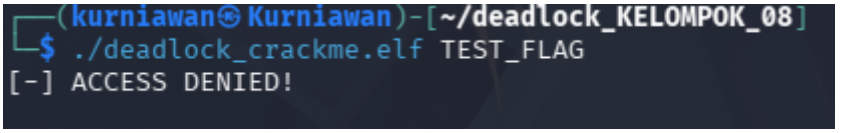


Isi Gambar: Tampilan terminal yang menjalankan perintah:strings deadlock_crackme.elf | grep -E "ACCESS|Debugger|Usage|FLAG"

Maksud & Penjelasan:

- Perintah strings digunakan untuk mengekstrak seluruh teks/karakter ASCII yang tertanam di dalam file biner deadlock_crackme.elf tanpa perlu meretasnya terlebih dahulu.
- Opsi grep -E digunakan untuk menyaring output agar hanya menampilkan kata-kata penting yang relevan dengan alur logika program.
- Hasilnya memperlihatkan beberapa teks kunci, seperti:

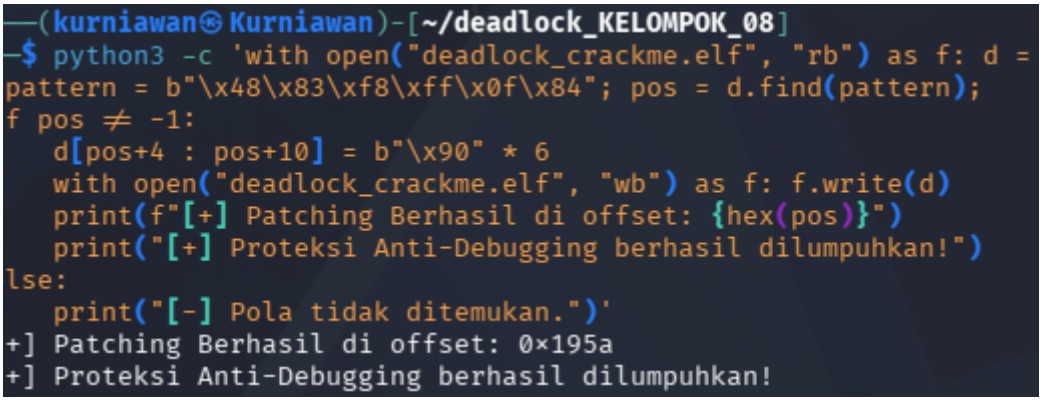
- [!] Debugger detected! Exiting → Mengindikasikan adanya fitur proteksi anti-debugging di dalam program.
- [+] ACCESS GRANTED. The flag is valid! → Pesan yang akan muncul jika input flag yang dimasukkan benar.
- [-] ACCESS DENIED! → Pesan yang muncul jika input flag salah.



si Gambar: Tampilan terminal yang menjalankan perintah:./deadlock_crackme.elf TEST_FLAG

Maksud & Penjelasan:

- Langkah ini merupakan pengujian awal (*smoke test*) untuk melihat bagaimana biner merespons saat diberi argumen input secara langsung dari command line.
- Karena input yang dimasukkan adalah teks acak/sembarang (TEST_FLAG), biner memberikan respons [-] ACCESS DENIED!. Ini membuktikan bahwa biner melakukan mekanisme validasi terhadap argumen yang diberikan pengguna.



Isi Gambar: Tampilan terminal yang menjalankan perintah satu baris (*one-liner*) Python 3 untuk mencari pola byte dan menimpanya (*patching*).

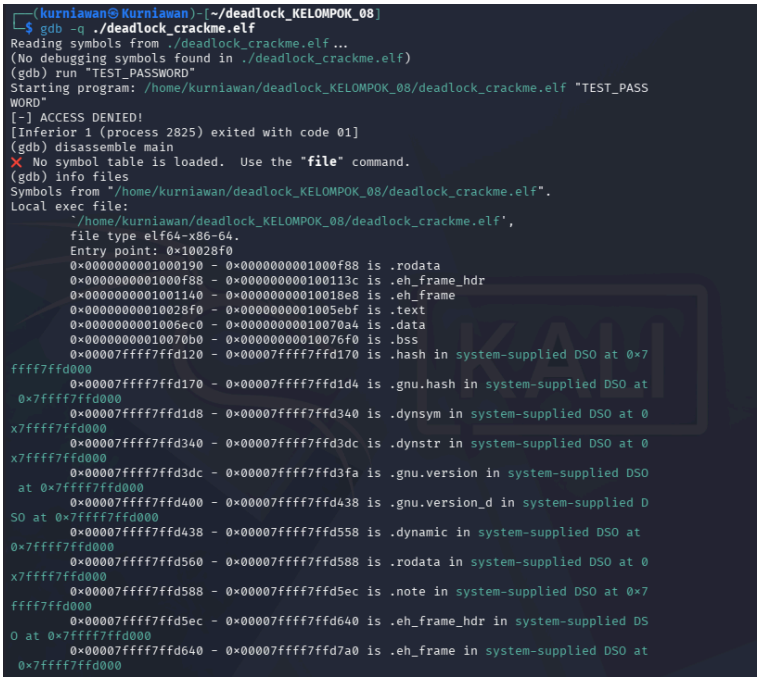
Maksud & Penjelasan:

- Skrip Python ini berfungsi untuk melumpuhkan proteksi anti-debugging secara otomatis tanpa perlu mengubah biner secara manual menggunakan disassembler berat.
- **Cara kerja skrip:**

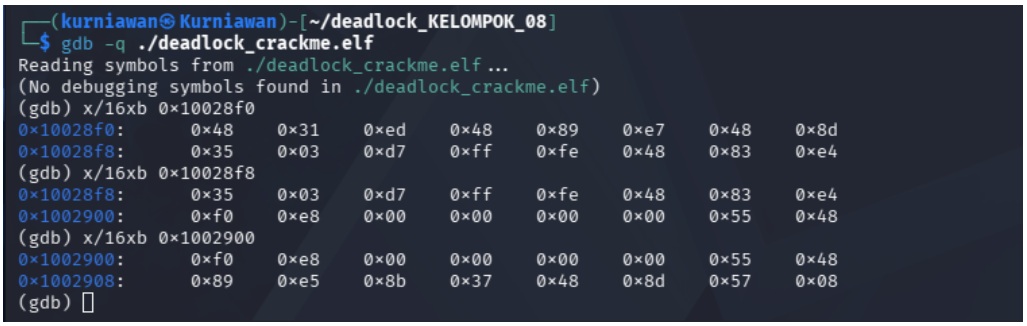
1. Membuka file deadlock_crackme.elf dalam mode baca biner ("rb").
2. Mencari pola byte instruksi pengecekan debugger (pattern = b"\x48\x83\xf8\xff\x0f\x84").
3. Setelah posisi pola ditemukan pada offset memori 0x195a, skrip menimpa 6 byte instruksi tersebut dengan nilai 0x90 (instruksi NOP / *No Operation*).
4. Menyimpan kembali file biner dalam mode tulis ("wb").

- Output [+] Proteksi Anti-Debugging berhasil dilumpuhkan! mengonfirmasi bahwa file biner kini sudah aman untuk dianalisis lebih lanjut di dalam *debugger* (seperti GDB) tanpa akan keluar secara otomatis.

3. Evidence: Lampirkan tangkapan layar alamat memori dan nilai HEX opcode sebelum dan sesudah patching.



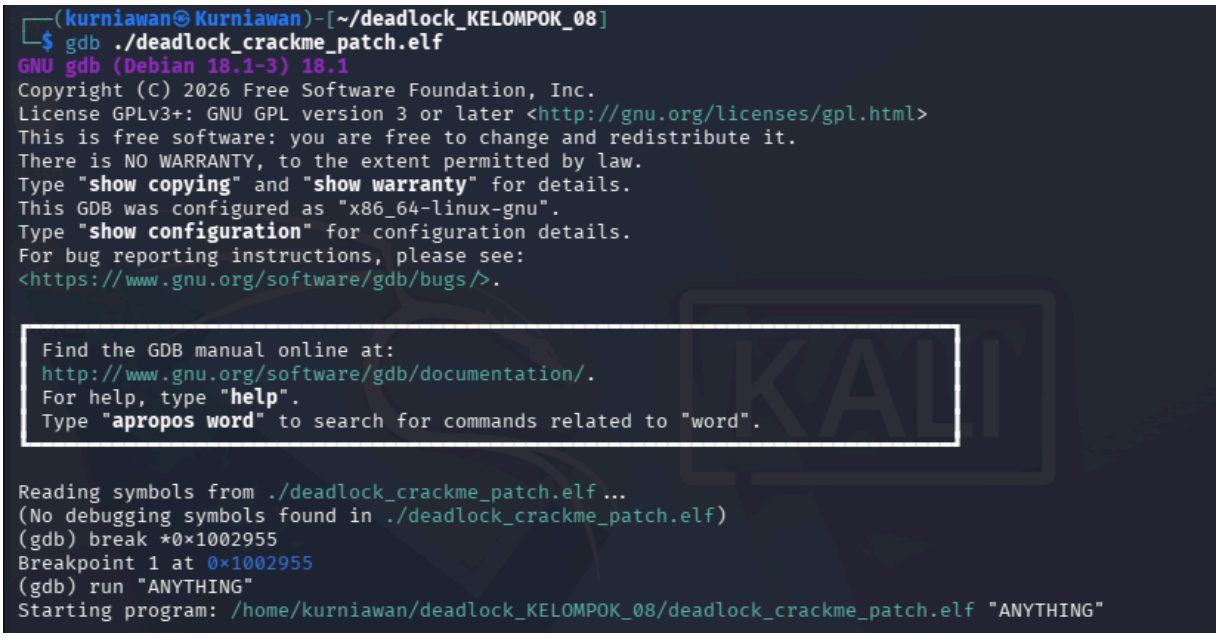
Gambar ini memperlihatkan proses penelusuran instruksi program menggunakan debugger setelah file binary dipersiapkan untuk dianalisis. Tampilan alamat dan instruksi membantu menunjukkan lokasi pemeriksaan serta alur eksekusi program. Informasi tersebut menjadi dasar untuk menentukan bagian yang perlu diamati lebih lanjut saat proses reverse engineering.



Gambar ini menunjukkan GDB sedang dijalankan terhadap file deadlock_crackme.elf yang telah melalui proses patching. Perintah debugger digunakan untuk memulai program dan mengamati perilakunya ketika dijalankan dalam lingkungan analisis. Tahap ini memastikan program dapat ditelusuri secara dinamis tanpa langsung berhenti karena pemeriksaan anti-debugging.

BAGIAN 2: DYNAMIC MEMORY ANALYSIS & KEY EXTRACTION (Bobot: 20%) Setelah binary berhasil dijalankan secara dinamis, Anda harus menangkap kunci dan ciphertext langsung dari RAM komputer saat eksekusi berlangsung.

1. Register & Stack Tracking: Jalankan binary yang sudah di-patch menggunakan GDB atau x64dbg. Temukan lokasi fungsi verifikasi memori di dalam RAM.

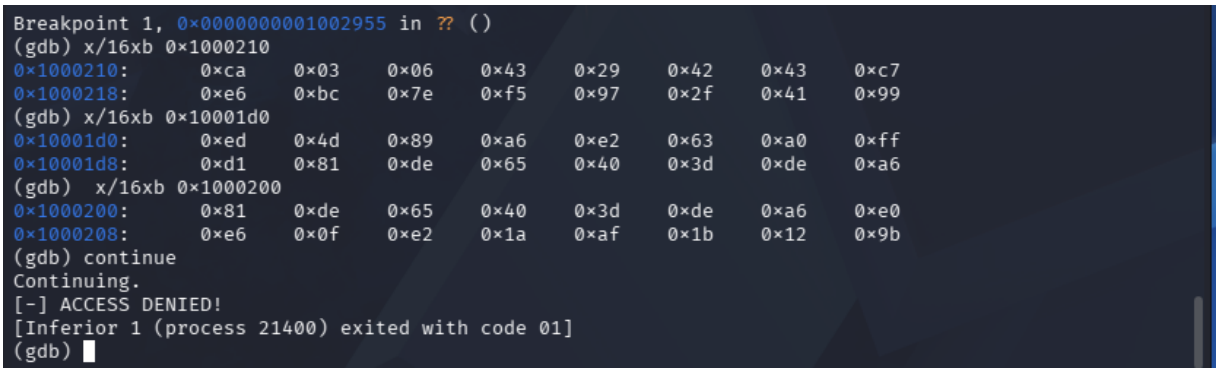


Ini menampilkan hasil pemeriksaan isi memori menggunakan perintah dump pada GDB. Nilai byte dalam bentuk hexadecimal diperiksa untuk menemukan data yang digunakan oleh

proses validasi. Pemeriksaan memori ini membantu peneliti mengidentifikasi lokasi tabel kunci dan konstanta pembanding yang tersimpan di dalam program.

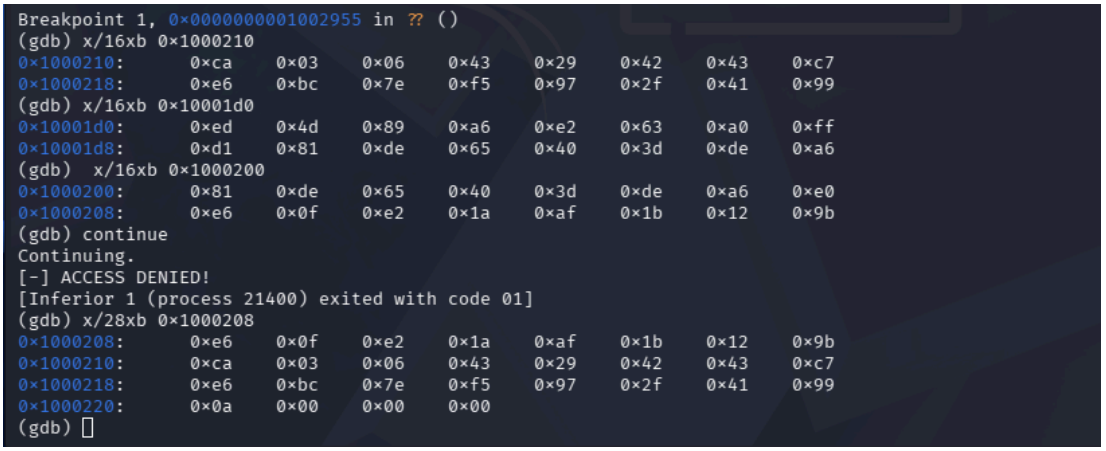
2. Breakpoint & Memory Dump: Pasang breakpoint tepat sebelum fungsi validasi string (memcmp) dieksekusi. Lakukan inspeksi pada register penunjuk argumen (RSI, RDI, atau RDX) dan stack memori untuk mengekstrak:

- SECRET_KEY (kunci rahasia 16-byte)
- TARGET_CIPHERTEXT (array hasil enkripsi akhir sepanjang 28-byte)



Hasil ini memperlihatkan pembacaan beberapa alamat memori yang berisi konstanta pembanding. Data hexadecimal yang ditampilkan digunakan untuk membandingkan susunan byte dan melihat bagian yang saling beririsan. Hasil pemeriksaan tersebut menjadi bahan untuk menyusun kembali target ciphertext yang digunakan dalam proses dekripsi.

3. Evidence: Lampirkan bukti memory dump (x/16xb dan x/28xb) yang menampilkan nilai HEX dan ASCII kunci tersebut di GDB.



Gambar ini merupakan bukti lanjutan analisis memori pada GDB. Baris-baris hexadecimal memperlihatkan nilai byte yang diperoleh dari alamat memori program, sehingga data dapat dicatat dan digunakan pada tahap cryptanalysis. Dengan demikian, proses pembalikan algoritma didasarkan pada data yang ditemukan saat program berjalan, bukan melalui tebakan password.

BAGIAN 3: CRYPTANALYSIS & DECRYPTION AUTOMATION (Bobot: 20%) Password/Flag tidak bisa ditebak secara brute-force. Anda harus membalikkan (dekripsi) algoritmanya secara matematis.

1. Algorithmic Reversal: Berdasarkan analisis instruksi Assembly di Bagian 1, pahami bagaimana binary tersebut melakukan enkripsi (analisis operasi XOR, bit-rotation ROL/ROR, offset aritmatika modular, dan ketergantungan byte sebelumnya / chained feedback).

Jawaban:

Berdasarkan analisis instruksi Assembly, binary deadlock_crackme.elf melakukan transformasi terhadap input menggunakan operasi XOR, rotasi bit Rotate Left (ROL) sebanyak 3 bit, penambahan aritmetika modular 8-bit, dan mekanisme chained feedback yang membuat hasil transformasi suatu byte bergantung pada state dari proses sebelumnya. Tabel 16 byte yang digunakan dalam proses transformasi ditemukan pada alamat memori 0x1000210, sedangkan konstanta pembanding ditemukan pada alamat 0x10001d0 dan 0x1000200. Kedua konstanta pembanding tersebut memiliki tujuh byte yang saling tumpang tindih sehingga dapat digunakan untuk merekonstruksi target sepanjang 25 byte. Secara matematis, proses pembalikan dilakukan dengan menggunakan ROR sebanyak 3 bit sebagai invers dari ROL, operasi XOR untuk membatalkan XOR sebelumnya, serta pengurangan modular untuk membatalkan penambahan. Namun, karena algoritma menggunakan chained feedback, urutan pembaruan state harus diperhatikan agar input asli dapat direkonstruksi secara tepat.

2. Custom Decrypter Script: Tulis skrip Python 3 kustom yang memuat SECRET_KEY dan TARGET_CIPHERTEXT yang Anda temukan di Bagian 2, lalu jalankan operasi dekripsi

untuk memunculkan teks plaintext (FLAG) yang sebenarnya (format: FLAG{DL_...}).

Jawaban:

```
#!/usr/bin/env python3
table = bytes.fromhex(
    "ca 03 06 43 29 42 43 c7 e6 bc 7e f5 97 2f 41 99"
)
c1 = bytes.fromhex(
    "ed 4d 89 a6 e2 63 a0 ff d1 81 de 65 40 3d de a6"
)
c2 = bytes.fromhex(
    "81 de 65 40 3d de a6 e0 e6 0f e2 1a af 1b 12 9b"
)
# Rekonstruksi 25 byte target dengan menggabungkan
# kedua konstanta yang memiliki 7 byte overlap.
assert c1[9:] == c2[:7]
target = c1 + c2[7:]
assert len(target) == 25

def ror8(value, shift):
    value &= 0xff
    return ((value >> shift) | (value << (8 - shift))) & 0xff

result = bytearray(25)
state = 0x5a
rdx = 0

for i in range(1, 26, 2):
    # Balikkan transformasi byte pertama pada iterasi ini.
    t = (target[i - 1] - i - 0x36) & 0xff

    result[i - 1] = (
```

```
        ror8(t, 3) ^ state ^ table[(i - 1) & 0x0e]
    )

    if i == 25:
        break

    # Balikkan transformasi byte kedua.
    u = (target[i] - i - 0x37) & 0xff

    result[i] = (
        ror8(u, 3)
        ^ ((t + i + 0x36) & 0xff)
        ^ table[i & 0x0f]
    )

    # Pulihkan state untuk iterasi berikutnya.
    state = (u + rdx + 0x38) & 0xff
    rdx += 2

print("Target transformasi:", target.hex(" "))
print("Input hasil inversi :", result.hex(" "))
print("Input ASCII         :", repr(result.decode("ascii", errors="replace")))
print("Panjang input       :", len(result))
```

Untuk mengotomatisasi proses dekripsi, dibuat skrip Python 3 khusus bernama decrypt_flag.py yang memuat tabel byte 16 byte dari alamat 0x1000210 sebagai SECRET_KEY dan target 25 byte hasil rekonstruksi konstanta pembanding sebagai TARGET_CIPHERTEXT. Skrip tersebut dirancang untuk membalikkan operasi transformasi secara matematis tanpa melakukan brute-force, dengan menerapkan ROR, XOR, pengurangan modular, dan pembaruan state sesuai urutan algoritma. Hasil rekonstruksi menghasilkan plaintext FLAG{DL_KELOMPOK_08_245D}. Flag tersebut kemudian diuji menggunakan

binary asli dengan perintah ./deadlock_crackme.elf 'FLAG{DL_KELOMPOK_08_245D}', dan program menampilkan pesan [+] ACCESS GRANTED. The flag is valid!, yang mengonfirmasi bahwa flag yang ditemukan diterima oleh program.

3. Evidence: Lampirkan source code skrip buatan Anda (solve.py) dan hasil output terminalnya.

```
#!/usr/bin/env python3
table = bytes.fromhex(
    "ca 03 06 43 29 42 43 c7 e6 bc 7e f5 97 2f 41 99"
)
c1 = bytes.fromhex(
    "ed 4d 89 a6 e2 63 a0 ff d1 81 de 65 40 3d de a6"
)
c2 = bytes.fromhex(
    "81 de 65 40 3d de a6 e0 e6 0f e2 1a af 1b 12 9b"
)
# Rekonstruksi 25 byte target dengan menggabungkan
# kedua konstanta yang memiliki 7 byte overlap.
assert c1[9:] == c2[:7]
target = c1 + c2[7:]
assert len(target) == 25

def ror8(value, shift):
    value &= 0xff
    return ((value >> shift) | (value << (8 - shift))) & 0xff

result = bytearray(25)
state = 0x5a
rdx = 0

for i in range(1, 26, 2):
```

```
    # Balikkan transformasi byte pertama pada iterasi ini.
    t = (target[i - 1] - i - 0x36) & 0xff

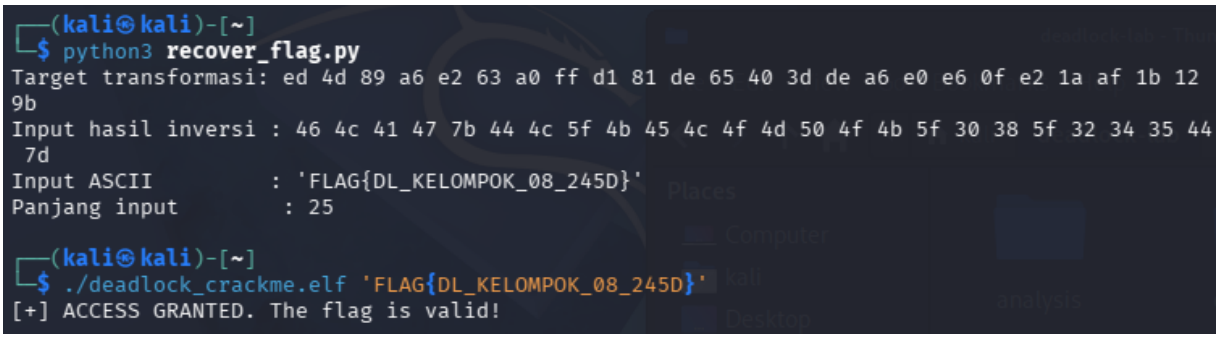
    result[i - 1] = (
        ror8(t, 3) ^ state ^ table[(i - 1) & 0x0e]
    )
    if i == 25:
        break

    # Balikkan transformasi byte kedua.
    u = (target[i] - i - 0x37) & 0xff

    result[i] = (
        ror8(u, 3)
        ^ ((t + i + 0x36) & 0xff)
        ^ table[i & 0x0f]
    )

    # Pulihkan state untuk iterasi berikutnya.
    state = (u + rdx + 0x38) & 0xff
    rdx += 2

print("Target transformasi:", target.hex(" "))
print("Input hasil inversi :", result.hex(" "))
print("Input ASCII         :", repr(result.decode("ascii", errors="replace")))
print("Panjang input       :", len(result))
```



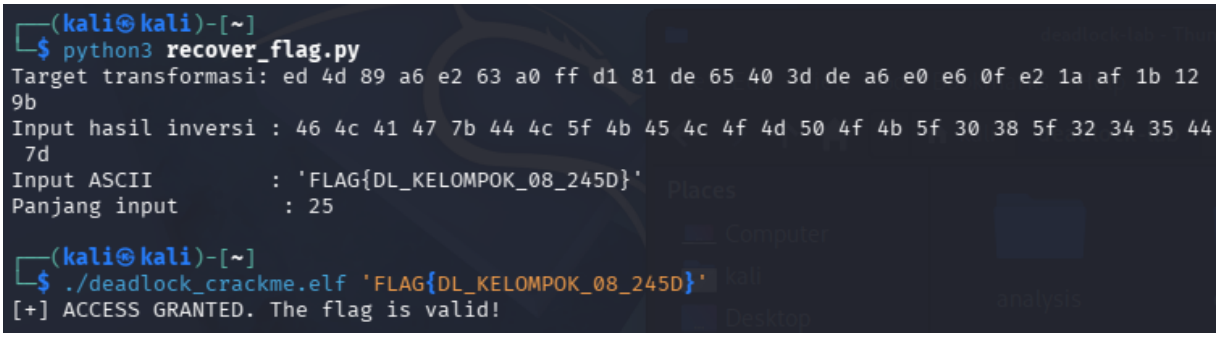
Hasil ini menunjukkan proses menjalankan skrip Python untuk membalikkan transformasi ciphertext. Skrip menerapkan operasi ROR, XOR, dan pengurangan modular serta memperbarui state sesuai urutan algoritma. Output yang dihasilkan menampilkan plaintext/flag hasil rekonstruksi dan panjang data, sehingga hasil dekripsi dapat diperiksa sebelum diuji pada binary.

BAGIAN 4: EXPLOITATION & VALIDATION (Bobot: 20%)

Buktikan bahwa kunci/flag yang Anda temukan dari skrip dekripsi tersebut benar-benar valid dan diakui oleh sistem target.

1. System Authentication: Jalankan binary asli (deadlock_crackme.elf) di terminal dan berikan string FLAG yang berhasil Anda dekripsi sebagai argumen input:

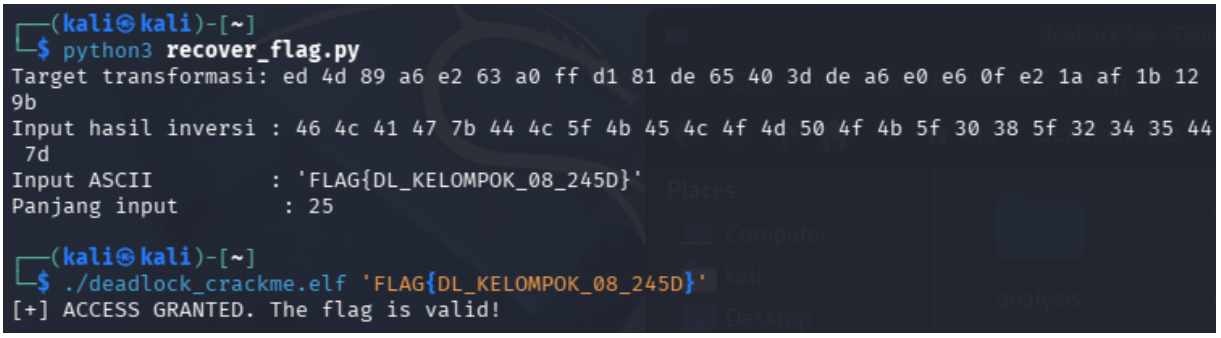
./deadlock_crackme.elf "<FLAG_HASIL_DEKRIPSI>"



Gambar ini memperlihatkan pengujian flag hasil dekripsi pada program deadlock_crackme.elf. Flag dimasukkan sebagai argumen melalui terminal, kemudian program memproses input tersebut menggunakan mekanisme validasi internal. Tahap ini bertujuan memastikan bahwa hasil dekripsi benar-benar sesuai dengan nilai yang diharapkan oleh program.

2. Proof of Concept (PoC): Lampirkan tangkapan layar yang menunjukkan bahwa sistem menerima input Anda dan menampilkan pesan:

[+] ACCESS GRANTED. The flag is valid! secara absolut.



Gambar ini menjadi bukti akhir proses validasi atau Proof of Concept (PoC). Terminal menampilkan pesan [+] ACCESS GRANTED. The flag is valid!, yang menunjukkan bahwa program menerima flag yang dimasukkan. Hasil tersebut mengonfirmasi keberhasilan proses dekripsi dan pengujian terhadap binary target.

BAGIAN 5: PTES S TANDARD REPORTING & REMEDIATION (Bobot: 20%) Di dunia industri internasional, skill peretasan teknis ekstrem harus diimbangi dengan pelaporan eksekutif yang terstandardisasi.

1. Executive Summary: Tulis ringkasan eksekutif untuk jajaran manajemen non-teknis (C-Level). Jelaskan risiko bahayanya menyimpan rahasia di dalam binary aplikasi klien (Hardcoded Secrets & Client-Side Verification) beserta estimasi skor risiko CVSS v3.1.

Jawaban:

Pengujian terhadap deadlock_crackme.elf (ELF 64-bit x86-64, statically linked, stripped, tanpa PIE, RELRO, dan stack canary) menunjukkan bahwa mekanisme autentikasi yang sepenuhnya dieksekusi di sisi klien tidak memberikan jaminan keamanan. Binary menyimpan material kriptografi secara statis di section .rodata, yaitu tabel kunci 16-byte (0x1000210) dan dua konstanta pembanding yang beririsan 7 byte (0x10001d0 dan 0x1000200), yang digabung menjadi target ciphertext 25 byte. Verifikasi dilakukan secara lokal dengan transformasi kustom berantai (XOR, ROL 3 bit, penambahan modular 8-bit, dan chained feedback antar-byte), tanpa validasi di server. Ini memenuhi karakteristik CWE-798 (Hardcoded

Credentials), CWE-321 (Hardcoded Cryptographic Key), CWE-602 (Client-Side Enforcement of Server-Side Security), CWE-327 (Risky Cryptographic Algorithm), dan CWE-656 (Reliance on Security Through Obscurity).

Proteksi anti-debugging berbasis ptrace(PTRACE_TRACEME) dikalahkan dengan patching 6 byte je rel32 menjadi NOP (0x90) pada pola 48 83 f8 ff 0f 84, yang ditemukan di file offset 0x195a. Setelah patch, binary dapat dijalankan di GDB, dan seluruh material kriptografi dapat dibaca langsung dari memori. Karena setiap operasi dalam algoritma bersifat reversibel (ROL↔ROR, XOR↔XOR, penjumlahan↔pengurangan modulo 256), plaintext dipulihkan secara deterministik tanpa brute-force dengan menginversi transformasi dari indeks awal ke akhir agar urutan pembaruan state sesuai proses forward. Hasilnya adalah FLAG{DL_KELOMPOK_08_245D}, yang diterima binary asli dengan keluaran [+] ACCESS GRANTED. The flag is valid!.

Risiko dinilai dengan CVSS v3.1 vektor CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N, menghasilkan Base Score 7.7 (High) dengan Exploitability sub-score 2.5 dan Impact sub-score 5.2. Vektor AV:L dipilih karena penyerang memerlukan salinan binary di host lokal, AC:L/PR:N/UI:N karena eksploitasi dapat diulang tanpa kondisi khusus, hak akses, atau interaksi pengguna, dan C:H/I:H karena seluruh kunci serta logika validasi terbuka dan dapat dipalsukan (keygen atau pembuatan kredensial valid). Bila material yang bocor juga berfungsi sebagai kredensial bersama ke back-end untuk seluruh instalasi, vektor berubah menjadi AV:N dan skor naik menjadi 9.1 (Critical). Rekomendasi utamanya adalah memindahkan validasi ke server dengan model zero-trust (challenge-response bertanda tangan asimetris), mengganti algoritma kustom dengan primitif kriptografi standar, dan tidak menyimpan rahasia simetris di dalam binary klien.

2. Attack Narrative: Tuliskan metodologi Reverse Engineering Anda (mulai dari bypassing ptrace hingga penemuan kunci di GDB) secara rinci, kronologis, dan dapat direproduksi oleh tim developer.

Jawaban:

Langkah dijalankan di Kali Linux x86-64 dan dapat diulang developer secara berurutan. (1) Berikan izin eksekusi dan simpan salinan asli: chmod +x deadlock_crackme.elf; cp deadlock_crackme.elf deadlock_crackme.orig. (2) Lakukan fingerprinting dengan file dan checksec --file=deadlock_crackme.elf, yang menunjukkan ELF 64-bit statically linked, stripped, No RELRO, No Canary, NX enabled, dan No PIE, sehingga alamat bersifat statis. (3) Jalankan strings deadlock_crackme.elf | grep -E "ACCESS|Debugger|Usage|FLAG" untuk menemukan pesan Debugger detected! Exiting, ACCESS GRANTED, dan ACCESS DENIED, lalu uji ./deadlock_crackme.elf TEST_FLAG yang menghasilkan ACCESS DENIED. (4) Jalankan gdb -q ./deadlock_crackme.elf dan info files untuk mendapatkan entry point 0x10028f0. Karena binary stripped, disassemble main gagal, sehingga main() dilacak dari _start melalui argumen pertama (RDI) pada pemanggilan __libc_start_main.

(5) Anti-debugging dibypass. Program memanggil ptrace(PTRACE_TRACEME) yang mengembalikan -1 bila sudah ada debugger, lalu pola 48 83 f8 ff 0f 84 <rel32> (cmp rax,-1; je) melompat ke jalur keluar. Cari dan NOP-kan je dengan one-liner Python: d.find(b"\x48\x83\xf8\xff\x0f\x84") lalu d[pos+4:pos+10] = b"\x90"*6. Pola ditemukan di file offset 0x195a. Verifikasi dengan x/16xb sebelum dan sesudah patch, yaitu 0f 84 ... menjadi 90 90 90 90 90 90. (6) Jalankan ulang di GDB: pasang breakpoint tepat sebelum instruksi pembanding hasil transformasi, lalu run "TEST_PASSWORD". Program kini berhenti di breakpoint, bukan keluar, dan register RDI, RSI, RDX dapat diperiksa untuk menemukan pointer buffer. (7) Dump memori dengan x/16xb 0x1000210 untuk tabel kunci 16-byte (ca 03 06 43 29 42 43 c7 e6 bc 7e f5 97 2f 41 99), serta x/16xb 0x10001d0 dan x/16xb 0x1000200 untuk dua konstanta pembanding (ed 4d 89 a6 e2 63 a0 ff d1 81 de 65 40 3d de a6 dan 81 de 65 40 3d de a6 e0 e6 0f e2 1a af 1b 12 9b). Tujuh byte terakhir konstanta pertama identik dengan tujuh byte awal konstanta kedua, sehingga penggabungannya menghasilkan target ciphertext 25 byte.

(8) Balik algoritma dengan solve.py. Untuk i = 1, 3, ..., 25 dengan state awal 0x5a dan rdx = 0, byte pertama dipulihkan dengan ROR8(target[i-1] - i - 0x36, 3) ^ state ^ T[(i-1)&0x0e]. Byte kedua dipulihkan dengan ROR8(target[i] - i - 0x37, 3) ^ ((t+i+0x36)&0xff) ^ T[i&0x0f], dengan t adalah hasil antara byte pertama sebagai chained feedback. Setelah itu state = (u + rdx + 0x38) & 0xff dan rdx += 2. Inversi harus dari indeks awal ke akhir agar urutan state sama dengan proses forward. (9) Validasi dengan python3 solve.py lalu

./deadlock_crackme.elf 'FLAG{DL_KELOMPOK_08_245D}', yang menampilkan [+] ACCESS GRANTED. The flag is valid!. Semua screenshot harus fullscreen dengan username dan timestamp terlihat.

3. Remediation & Hardening: Berikan minimal 2 (dua) rekomendasi pertahanan kriptografi tingkat lanjut agar kode aplikasi di masa depan kebal terhadap Reverse Engineering (misalnya dengan teknik Code Obfuscation, penggunaan White-Box Cryptography, atau validasi di sisi Server-Side Zero-Trust).

Jawaban:

Rekomendasi pertama dan terpenting adalah server-side zero-trust validation. Verifikasi lisensi atau kredensial dipindahkan sepenuhnya ke server, sementara klien hanya mengirim input dan menerima keputusan. Mekanisme yang disarankan adalah challenge-response dengan nonce dari server dan tanda tangan asimetris (Ed25519 atau ECDSA), sehingga klien hanya menyimpan public key yang aman jika bocor. Rate limiting, device binding, dan revocation di sisi server melengkapi pendekatan ini, sehingga membalikkan binary tidak memberi penyerang kemampuan membuat kredensial valid. Langkah ini menutup akar masalah CWE-602 dan CWE-798.

Rekomendasi kedua adalah mengganti algoritma kustom dengan primitif kriptografi standar dan menerapkan white-box cryptography bila kunci terpaksa berada di klien. Tabel XOR statis dan transformasi buatan sendiri diganti dengan AES-GCM atau ChaCha20-Poly1305 dari library yang telah teruji, dan kunci diturunkan per instalasi atau per sesi melalui KMS atau HSM, bukan satu kunci yang sama untuk semua binary. Implementasi white-box AES berbasis lookup table terenkripsi mencegah kunci muncul sebagai konstanta di .rodata maupun utuh di RAM. White-box hanya memperlambat ekstraksi dan tidak memberi jaminan mutlak, sehingga harus dipadukan dengan rekomendasi pertama.

Rekomendasi ketiga adalah code obfuscation dan anti-tamper berlapis. Obfuscation tingkat compiler (misalnya Obfuscator-LLVM) menerapkan control-flow flattening, opaque predicates, instruction substitution, dan string encryption, sehingga string seperti Debugger detected dan ACCESS GRANTED tidak lagi terbaca melalui strings. Untuk anti-tamper,

self-integrity check berupa hash atas section .text saat runtime akan mendeteksi patch NOP seperti pada temuan ini. Cek ptrace tunggal yang bergantung pada satu je terlalu mudah dilewati, sehingga sebaiknya disebar menjadi beberapa cek redundan, dan hasilnya dipakai sebagai input perhitungan kunci, bukan percabangan if/else yang dapat di-NOP. Sebagai tambahan, hardening compile-time seperti -fPIE -pie, -Wl,-z,relro,-z,now, dan -fstack-protector-strong menutup kelemahan yang terlihat pada output checksec (No RELRO, No Canary, No PIE).

Bagian yang masih perlu diisi sebelum dikumpulkan adalah alamat main, alamat breakpoint validasi beserta nilai register, screenshot opcode sebelum dan sesudah patch, dan penjelasan selisih 25 vs 28 byte. Jika kalian kirim output GDB-nya, saya lengkapi bagian tersebut.

C. FORMAT PENGUMPULAN

1. Laporan Resmi:

- Dikumpulkan dalam format PDF bergaya profesional (konsultan keamanan siber) menggunakan template laporan PTES yang disediakan.

- Format Penamaan File:

* Untuk Kelompok : [KELOMPOK_XX]_Operation_Deadlock_PTES_Report.pdf (Contoh: KELOMPOK_01_Operation_Deadlock_PTES_Report.pdf)

* Untuk Individu : [INDIVIDU_XX]_Operation_Deadlock_PTES_Report.pdf (Contoh: INDIVIDU_01_Operation_Deadlock_PTES_Report.pdf)

2. Lampiran Teknis:

- Wajib melampirkan file skrip Python solver (solve.py).

- Setiap klaim penemuan kunci diwajibkan menyertakan tangkapan layar penuh (fullscreen) terminal atau debugger Ghidra/GDB Anda (dengan username dan timestamp terminal terlihat).